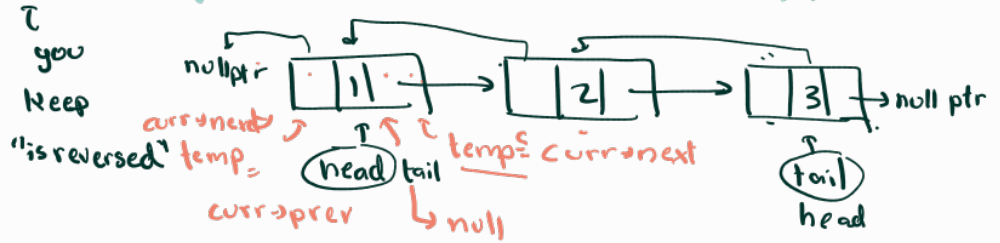


\* add vectors, arrays, OOP \*

1) What is the Big-O notation time complexity (worst case) of an in-place reversal of a doubly linked list?  $O(N)$ . What if you could swap the direction flags (head and tail sentinels)?  $O(1)$

- A)  $O(1)$ ,  $O(N)$   
 (B)  $O(N)$ ,  $O(1)$   
 C)  $O(N)$ ,  $O(N)$   
 D)  $O(1)$ ,  $O(1)$



$O(N)$  ← since we visit each one and swap

space  $O(1)$  ← just temp variables

focus on part 1

week 6 png - 1-10 array  
 ↳ stack

2. In a circularly linked list with a 'cursor', the following code is used to insert a new node (which should be pointed to by 'cursor' after insertion). What best fills the missing line?

```
template <typename T>
void addNode(T data) {
```

```
    ListNode* newNode = new ListNode(data);
```

```
    if (!cursor) {
```

```
        cursor = newNode;
```

```
        cursor->next = cursor;
```

```
        return;
```

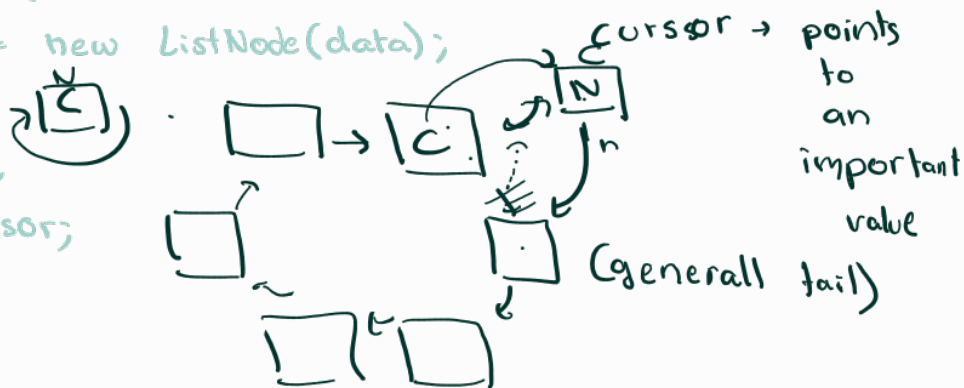
```
    }
```

```
    newNode->next = cursor->next;
```

```
    // MISSING ← cursor->next = newNode;
```

```
    cursor = newNode;
```

```
}
```

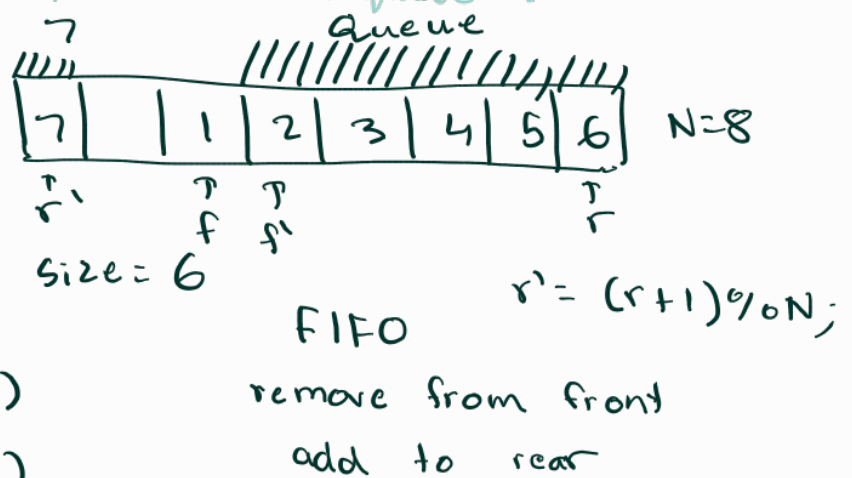


newnode → next = cursor → next

- A)  $\text{cursor} \rightarrow \text{next} = \text{cursor};$
- B)  $\text{head} \rightarrow \text{next} = \text{newNode};$
- C)  $\text{cursor} \rightarrow \text{next} = \text{newNode};$**
- D)  $\text{newNode} \rightarrow \text{next} = \text{cursor};$

3. An array of size  $N$  is used to implement a Queue (circular array), the implementation uses 'front' and 'rear' indices. If the queue is neither empty nor full, what are the independent time complexities of 'enqueue' and 'dequeue'?

- A)  $O(1)$  ,  $O(N)$
- B)  $O(N)$  ,  $O(1)$
- C)  $O(1)$  ,  $O(1)$**
- D)  $O(N)$  ,  $O(N)$



en  $O(1)$   
 de  $O(1)$

4. Which of the following scenarios is a deque <sup>"deck"</sup> least optimally suited for than a standard stack or queue? insert/remove both ends
- good A) undo/redo history, items pushed and popped from both ends
- good B) sliding window maximum algorithm (long answer problem\*)
- bad C) strict Last-In-First-Out (LIFO) recursive function call stack
- good D) browser history - add/remove from front and back

5. What are the values of X and Y after the following pseudocode executes on an initially empty stack S?

- 1 S.push(10)
- 2 S.push(20)
- 3 X = S.pop() // assuming pop returns popped value
- 4 S.push(30)
- 5 S.push(X)
- 6 Y = S.pop()

0. S = [] <sup>access + insertion end</sup>

1. S = [10]

2. S = [10, 20]

3. X = 20 ; S = [10]

4. S = [10, 30]

5. S = [10, 30, 20]

6. Y = 20 ; S = [10, 30]

× A) X = 10, Y = 20

☒ B) X = 20, Y = 20

× C) X = 20, Y = 30

× D) X = 10, Y = 30

Usually, you'd have more MCQs, but this is a review session

I'd definitely recommend reading the textbook and its problems

For the sake of time, we'll move onto the free response questions.

(usually, expect 10 or 20 MCQs)

6. Given a string  $s$  containing just the characters '(', ')', '{', '}', '[', ']', write a function to determine if the string is valid.

Valid if:

- open brackets closed by same type closing brackets / if else statements
- open brackets closed in the correct order / stack
- all open brackets are closed / check at end if any open remain

valid:  $s = "{([ ]})}"$

invalid:  $s = "{([ ])"$

↳ LIFO  $\rightarrow$  stack

method: 1. keep track of open brackets in order (newest first out)  
2. for each  $ch$  in  $s$ , check bracket type + compatibility

code is already on canvas

bool isValid(string s) {

stack<char> st;

for (char ch : s) {

if (ch == '(' || ch == '{' || ch == '[') {

st.push(ch);

else { if (st.empty()) return false;  
char top = st.top();

if ((ch == ')' && top != '(') ||

(ch == '}' && top != '{') ||

(ch == ']' && top != '[')) {

return false;

}

}

}

if (!st.empty()) return false; } return st.empty();

return true;

}

7. Given a string  $s$ , a duplicate removal consists of choosing two adjacent and equal letters and removing them. We repeatedly make duplicate removals on  $s$  until we no longer can. Return the reversed final string. (added "reversed" for time's sake)

Ex:  $s = \text{"abbaca"}$       stack  $\rightarrow$  match with last  
output: "ac"

why: "abbaca"  $\rightarrow$  "aaca"  $\rightarrow$  "ca"  $\rightarrow$  "ac"

```
string removeDups(string s) {
```

```
    stack<char> st;
```

```
    for (char ch : s) {
```

```
        if (!st.empty() && st.top() == ch) {  
            st.pop();
```

```
        } else {
```

```
            st.push(ch);
```

```
        }
```

```
    }
```

```
    string res = "";
```

```
    while (!st.empty()) {
```

```
        res += st.top();
```

```
        st.pop();
```

```
    }
```

```
    return res;
```

```
}
```

7. You are given a Singly Linked List - You need to reverse the list in-place (without allocating memory for new nodes)

Ex:  $1 \rightarrow 2 \rightarrow 3 \rightarrow \text{NULL}$  to  $3 \rightarrow 2 \rightarrow 1 \rightarrow \text{NULL}$

ListNode\* reverseLinkedList(ListNode\* head) {

ListNode\* prev = nullptr;

ListNode\* curr = head;

ListNode\* next = nullptr;

while (curr != nullptr) {

next = curr->next; // store

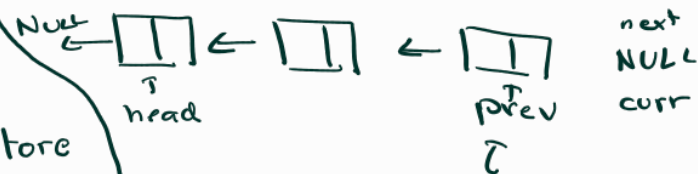
curr->next = prev; // reverse

prev = curr; // update prev

curr = next; // update curr using next

} // all the reversing

return prev; // points to new head



has new head  
when curr goes  
to NULL.

8. You are processing a dynamic stream of lowercase English letters one by one. You need to design an algorithm that can continuously output the first non-repeating character seen so far in the stream. If at any point there is no unique character, output '#'

$'a' - 'z' = 26$     index = key  
                                          value = value

earliest  $\rightarrow$  FIFO

ex: stream = "a b a b"    output = "a a b #"

0   1   2   3

0. Unique : [a]    output : 'a'  
               earliest non repeated
1. Unique : [a, b]    output : 'a'
2. Unique : [b, a]    output : 'b'
3. Unique : [~~a~~]    output : '#'  
               [~~b~~, ~~a~~, ~~b~~]

queue  $\rightarrow$  order of unique

vector  $\rightarrow$  freq of letters

1. Keep track of what we've seen using queue, unique

2. Keep track of frequencies of seen letters

```

string findUniq(string stream) {
    queue<char> q; // tracks order of unique characters
    vector<int> freq(26, 0); // tracks frequencies of 'a'-'z'
    string result = "";

    for (char ch : s) {
        q.push(ch);
        freq[ch - 'a']++; // update freq tracker
        while (!q.empty() && freq[q.front() - 'a'] > 1) {
            q.pop();
        }
        if (q.empty()) {
            result += '#'; // no unique at this point
        } else {
            result += q.front(); // earliest unique so far
        }
    }
    return result;
}
  
```



9. You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window. <sup>Arr of largest in each window</sup>

.z → sends output

a) from front → remove out of bounds

Input : `nums = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3`

b) from back → remove smaller values

Output : `[3, 3, 5, 5, 6, 7]` // maximum for each window

```
vector<int> maxWindow(vector<int>& nums, int k) {
    vector<int> result;
    deque<int> dq; // store indices

    for (int i = 0; i < nums.size(); ++i) {
        if (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front(); // hopefully exists // front always max
        } // remove indices outside window

        while (!dq.empty() && nums[dq.back()] <= nums[i]) {
            dq.pop_back();
        } // remove smaller from back - never max

        dq.push_back(i);

        if (i >= k - 1) {
            result.push_back(nums[dq.front()]);
        } // front of dq is current window max
    } // start recording after window is created

    return result;
}
```

end of review session ~

(once again, you'll do great)

Fin ... or is it?





(and vectors, in a way)

10. Arrays: Search a 2D matrix

given integer 'matrix' ( $m \times n$ ), each row decreasing, first <sup>non</sup> of each row  $>$  prev first

given target, true if target in matrix

$O(\log(m \times n))$  time complexity projecting binary search to 2D array

```
bool searchMatrix (vector<vector<int>> &matrix, int target) {
```

```
    int row = matrix.size(); // m - number of rows
```

```
    int col = matrix[0].size(); // n - number of columns
```

```
    int start = 0;
```

```
    int end = row * col - 1; // end of array
```

```
    while (start <= end) {
```

```
        int mid = start + (end - start) / 2; // binary search
```

```
        int r = mid / col;
```

```
        int c = mid % col;
```

```
        int val = matrix[r][c];
```

```
        if (val == target) {
```

```
            return true;
```

```
        } else if (val < target) {
```

```
            start = mid + 1;
```

```
        } else {
```

```
            end = mid - 1;
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

// we turn  $arr[m][n]$  into  $arr[m * n]$ , work out

// the identifiers, then do regular binary search

So, turns out I don't have enough time, so I'd suggest looking through the github repo and understanding the examples there. Reach out to me with any specific questions.

All the best 🍀